

codeHive

Algorithms & Recursion

A Deep Dive into the Fibonacci Sequence

February 2026 Edition

1. A Simple Algorithm: Fibonacci

The Fibonacci numbers are an excellent way to introduce core algorithmic concepts. Named after a 13th-century Italian mathematician, the sequence follows a simple but powerful rule.

The Rule: The first two numbers are 0 and 1. Every subsequent number is the sum of the two preceding ones.

The sequence looks like this:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Why Fibonacci?

- It demonstrates **State Management** (keeping track of previous values).
- It is the perfect playground to compare **Loops** vs. **Recursion**.
- It helps visualize **Growth Complexity**.

2. How the Algorithm Works

Before we write code, we define the logical steps (the algorithm). To generate the first 20 Fibonacci numbers:

1. **Initialize:** Start with two variables representing 0 and 1.
2. **Add:** Add the two previous numbers to create the new one.
3. **Update:** Shift the values (The "new" becomes "previous 1", and "previous 1" becomes "previous 2").
4. **Repeat:** Perform these steps 18 more times (to reach 20 total).

Thinking in Code

While the mathematical definition is fixed, the *implementation* can vary significantly. We will explore three distinct approaches.

codeHive

3. Implementation: Using a For Loop

The iterative approach is usually the most efficient way to generate the sequence for large ranges. It uses a for loop to traverse the range and update variables linearly.

Python: Iterative Method

```
prev2 = 0
prev1 = 1

print(prev2)
print(prev1)

for fibo in range(18):
    newFibo = prev1 + prev2
    print(newFibo)
    prev2 = prev1
    prev1 = newFibo
```

Observation: This method is very fast because we only calculate each number once. We use minimal memory (only storing three integers at a time).

4. Implementation: Basic Recursion

Recursion is a technique where a function calls itself. Instead of a loop, we use a stopping condition (base case) and function calls to "repeat" the logic.

Python: Recursive Function

```
print(0)
print(1)
count = 2

def fibonacci(prev1, prev2):
    global count
    if count ≤ 19:
        newFibo = prev1 + prev2
        print(newFibo)
        prev2 = prev1
        prev1 = newFibo
        count += 1
        fibonacci(prev1, prev2) # The function calls itself
    else:
        return

fibonacci(1, 0)
```

Notice that the function replaces the for loop by calling itself until the count reaches 19.

5. Finding the n-th Fibonacci Number

A common interview question is "Find the n-th Fibonacci number." This relies on the mathematical definition:

$$F(n) = F(n - 1) + F(n - 2)$$

Python: Mathematical Recursion

```
def F(n):  
    if n ≤ 1:  
        return n  
    else:  
        return F(n - 1) + F(n - 2)  
  
print(F(19)) # Outputs the 20th Fibonacci number
```

The Performance Trap

This specific recursive method is **inefficient**. Why? Because the number of function calls doubles with every increase in n. For example, to find F(10), the computer calculates F(5) many times unnecessarily.

6. Summary & Comparison

As you progress in DSA, choosing the right implementation is as important as choosing the right algorithm.

Method	Logic Type	Performance	Best For...
For Loop	Iterative	Fast / Efficient	General use, large numbers
Tail Recursion	Recursive	Moderate	Functional programming styles
Math Recursion	Recursive	Very Slow	Demonstrating theory

Key Takeaway: An algorithm can be implemented in many ways. While recursion is elegant and mathematically "pure," loops are often more practical for computer hardware.

7. Test Your Progress

Exercise: The Recursive Gap

Complete the code to make the `fibonacci()` function call itself for the next set of numbers:

```
def fibonacci(prev1, prev2):  
    if count ≤ 19:  
        newFibo = prev1 + prev2  
        # ... update variables ...  
        _____(prev1, prev2) # Hint: Put function name here
```

Answer: `fibonacci(prev1, prev2)`

© 2026 codeHive | Digital Learning Series

Proceed to Chapter 3: Mastering Arrays